Detailed Manual: Programming the PS (ARM Processor) Using SDK on PYNQ-Z2

Board

- PYNQ-Z2
- Zynq-7000 SoC
- ARM Cortex-A9 + FPGA fabric

---

1. Understanding the Architecture

The Zynq device contains:

PS (Processing System)

This is the ARM processor section.

It contains:

- ARM Cortex-A9 CPU
- DDR controller
- UART
- USB
- Ethernet
- Timers
- GPIO

The PS runs:

- C programs
- baremetal applications
- Linux
- embedded software

PL (Programmable Logic)

This is the FPGA section.

The PL contains:

- custom Verilog/VHDL modules
- DSP blocks
- BRAM
- AXI peripherals

Examples:

- adders
- multipliers
- CNN accelerators
- image processing pipelines

Communication Between PS and PL

Communication happens through:

- AXI interconnect
- AXI GPIO
- AXI DMA
- AXI BRAM

For beginners, AXI GPIO is easiest.

---

2. Complete Development Flow

The complete flow is:

```
Vivado → Generate Hardware → Export Hardware → SDK → Write C Code → Generate ELF → Program FPGA → Run Software
```

---

3. Vivado Hardware Design Requirements

Before SDK can work correctly, Vivado hardware must contain:

Mandatory Components

1. Zynq Processing System

Without PS7 block:

- SDK will not work correctly
- bitstream generation may fail
- ARM processor cannot run

You must add:

- ZYNQ7 Processing System IP

Then:

- Run Block Automation

---

2. AXI Interconnect

Needed for:

- PS ↔ PL communication

Vivado usually creates this automatically.

---

3. AXI GPIO

Used for sending data between:

- ARM processor
- FPGA logic

Example:

- GPIO0 = input A
- GPIO1 = input B
- GPIO2 = output SUM

---

4. HDL Wrapper

After block design completion:

Right click:

```
Design Sources
→ design_1
→ Create HDL Wrapper
```

Select:

```
Let Vivado manage wrapper
```

Without wrapper:

- synthesis may work
- bitstream generation often fails

---

4. Common Vivado Errors and Fixes

ERROR: NSTD-1 Unspecified IO Standard

Example:

```
Unspecified I/O Standard
```

Cause:

- external FPGA pins are unconstrained

Fix:

- use block design instead of raw top module
- avoid exposing FPGA pins directly for PS/PL communication

OR

add XDC constraints manually.

ERROR: UCIO-1 Unconstrained Logical Port

Cause:

- ports connected directly to FPGA pins
- no LOC constraints

Fix:

- use AXI GPIO inside block design
- avoid top-level external ports unless required

ERROR: PS7 block required

Cause:

- design does not contain Zynq PS block

Fix:

- add ZYNQ7 Processing System IP

Bitstream Generation Failed

Possible causes:

Cause 1

No HDL wrapper.

Fix:

- create HDL wrapper

Cause 2

Design errors.

Fix:

- Validate Design

Cause 3

Unconnected AXI interfaces.

Fix:

- Run Connection Automation

Cause 4

Clock/reset not connected.

Fix:

- connect FCLK_CLK0
- connect peripheral reset

---

5. Exporting Hardware to SDK

After bitstream generation:

```
File
→ Export Hardware
```

IMPORTANT:

Enable:

```
Include Bitstream
```

Then:

```
File
→ Launch SDK
```

---

6. Understanding SDK

SDK is used for:

- writing C programs for ARM
- compiling software

- generating ELF files
- debugging software
- communicating through UART

SDK works on the PS side only.

---

7. What Happens When SDK Opens

When SDK launches, it imports hardware information from Vivado.

You may see:

```
ps7_cortexa9
ps7_uart
ps7_ddr
axi_gpio
```

This means:

- SDK successfully detected hardware

---

8. SDK Workspace Structure

Inside SDK workspace:

Hardware Platform Project

Example:

```
design_1_wrapper_hw_platform_0
```

Contains:

- bitstream info
- hardware addresses
- BSP generation

BSP Project

Example:

```
adder_app_bsp
```

Contains:

- drivers
- libraries
- xparameters.h

Application Project

Example:

```
adder_app
```

Contains:

- main.c
- user software

---

9. Creating New SDK Application

Step 1

```
File
→ New
→ Application Project
```

Step 2

Fill:

Project Name

```
adder_app
```

Hardware Platform

Choose exported hardware.

Processor

```
ps7_cortexa9_0
```

OS Platform

```
standalone
```

Language

```
C
```

Step 3

Choose template:

```
Empty Application
```

---

10. Why main.c Sometimes Does Not Exist

When selecting:

```
Empty Application
```

SDK creates:

- project structure only
- no source file

You must manually create:

```
main.c
```

---

11. Creating main.c

Right click:

```
src
→ New
→ Source File
```

Name:

```
main.c
```

---

12. Understanding BSP

BSP = Board Support Package

Contains:

- drivers
- UART support
- GPIO support
- xil_printf
- xparameters.h

Without BSP:

- platform.h errors occur

- xgpio.h errors occur

---

13. Common SDK Errors and Fixes

ERROR: platform.h not found

Cause:

- BSP corrupted
- BSP not generated

Fix:

Right click BSP:

```
adder_app_bsp
→ Re-generate BSP Sources
```

Then:

```
Project
→ Clean
Project
→ Build All
```

ERROR: xparameters.h not found

Cause:

- BSP not generated
- wrong include path

Fix:

- regenerate BSP

File location:

```
adder_app_bsp/system.mss
adder_app_bsp/include/xparameters.h
```

ERROR: undefined reference to XGpio

Cause:

- xgpio library missing

Fix:

- ensure AXI GPIO exists in hardware
- regenerate BSP

ERROR: scanf not working

Cause:

- UART terminal not connected
- wrong baud rate

Fix:

Use:

```
115200 baud
```

ERROR: terminal shows garbage characters

Cause:

- wrong baud rate

Fix:

Set:

```
115200
8 data bits
```

```
1 stop bit
No parity
```

ERROR: nothing appears in terminal

Possible causes:

Cause 1

Wrong COM port.

Check Device Manager.

Cause 2

FPGA not programmed.

Fix:

```
Xilinx Tools
→ Program FPGA
```

Cause 3

Application not running.

Fix:

```
Run As
→ Launch on Hardware
```

Cause 4

UART disabled in PS.

Fix:

- enable UART during PS block configuration

ERROR: SDK freezes while launching

Possible causes:

- JTAG issue
- USB driver issue
- board not powered

Fix:

- reconnect USB
- power cycle board
- reinstall Digilent cable drivers

---

14. Understanding xparameters.h

This file contains:

- peripheral addresses
- device IDs
- hardware configuration

Example:

```
#define XPAR_AXI_GPIO_0_DEVICE_ID 0
#define XPAR_AXI_GPIO_0_BASEADDR 0x41200000
```

Generated automatically from Vivado hardware.

---

15. Understanding AXI GPIO in Software

Example:

```
XGpio gpio_a;
```

Creates GPIO driver object.

Initialization

```
XGpio_Initialize(&gpio_a, XPAR_AXI_GPIO_0_DEVICE_ID);
```

Connects software driver to hardware peripheral.

Set Direction

Output

```
XGpio_SetDataDirection(&gpio_a, 1, 0x00000000);
```

0 means output.

Input

```
XGpio_SetDataDirection(&gpio_sum, 1, 0xFFFFFFFF);
```

1 means input.

Write Data

```
XGpio_DiscreteWrite(&gpio_a, 1, value);
```

Writes value from ARM to FPGA.

Read Data

```
sum = XGpio_DiscreteRead(&gpio_sum, 1);
```

Reads value from FPGA to ARM.

---

16. Build Process

After writing code:

```
Project
→ Build Project
```

SDK generates:

```
Debug/adder_app.elf
```

---

17. Understanding ELF File

ELF = Executable and Linkable Format

Contains:

- machine code
- compiled C program

Runs on:

- ARM Cortex-A9

Not on FPGA fabric.

---

18. Programming FPGA from SDK

In SDK:

```
Xilinx Tools
→ Program FPGA
```

Select:

```
design_1_wrapper.bit
```

This loads hardware into PL.

---

19. Running Software on ARM

Right click application:

```
Run As
→ Launch on Hardware
```

SDK now:

1. downloads ELF
2. initializes ARM
3. loads program into DDR
4. starts execution

---

20. UART Terminal Setup

Use:

- PuTTY
- TeraTerm

Settings:

```
Baud Rate = 115200
Data Bits = 8
Parity = None
Stop Bits = 1
Flow Control = None
```

---

21. Complete Example Program

```c
#include <stdio.h>
#include "xgpio.h"
#include "xparameters.h"

XGpio gpio_a;
```

```
XGpio gpio_b;
XGpio gpio_sum;

int main()
{
    u32 a, b, sum;

    XGpio_Initialize(&gpio_a, XPAR_AXI_GPIO_0_DEVICE_ID);
    XGpio_Initialize(&gpio_b, XPAR_AXI_GPIO_1_DEVICE_ID);
    XGpio_Initialize(&gpio_sum, XPAR_AXI_GPIO_2_DEVICE_ID);

    XGpio_SetDataDirection(&gpio_a, 1, 0x00000000);
    XGpio_SetDataDirection(&gpio_b, 1, 0x00000000);
    XGpio_SetDataDirection(&gpio_sum, 1, 0xFFFFFFFF);

    while(1)
    {
        printf("\nEnter A: ");
        scanf("%u", &a);

        printf("Enter B: ");
        scanf("%u", &b);

        XGpio_DiscreteWrite(&gpio_a, 1, a);
        XGpio_DiscreteWrite(&gpio_b, 1, b);

        sum = XGpio_DiscreteRead(&gpio_sum, 1);

        printf("SUM = %u\n", sum);
    }

    return 0;
}
```

---

22. Full Execution Flow

```
Keyboard Input
↓
UART
↓
ARM Processor (PS)
↓
AXI GPIO Write
↓
```

```
Adder in FPGA (PL)
↓
AXI GPIO Read
↓
ARM Processor
↓
UART
↓
Terminal Output
```

---

23. How to Verify Hardware Is Correct

Method 1

Check xparameters.h

If AXI GPIO exists:

```
XPAR_AXI_GPIO_0_DEVICE_ID
```

hardware is exported correctly.

Method 2

Check Address Editor in Vivado.

You should see:

- AXI GPIO addresses

Method 3

Program FPGA successfully.

---

24. Important Beginner Mistakes

Mistake 1

Using raw Verilog top module.

Better:

- use block design
- use AXI GPIO

Mistake 2

Not exporting bitstream.

Always check:

```
Include Bitstream
```

Mistake 3

Forgetting HDL wrapper.

Mistake 4

Wrong GPIO direction.

Output GPIO:

```
0x00000000
```

Input GPIO:

```
0xFFFFFFFF
```

Mistake 5

Wrong COM port.

---

25. What You Have Learned

You now understand:

- Vivado hardware design
- PS vs PL
- AXI communication
- SDK workflow
- BSP
- ELF generation
- UART communication
- ARM software execution
- FPGA programming
- PS/PL interaction

---

26. Next Topics to Learn

After mastering AXI GPIO, learn:

1. AXI DMA
2. Interrupts
3. AXI Stream
4. BRAM interface
5. Linux on Zynq
6. PYNQ Python overlays
7. High Level Synthesis (HLS)
8. Custom AXI IP design
9. Video processing pipelines
10. CNN accelerators

These are heavily used in:

- AI accelerators
- computer vision
- robotics
- embedded systems
- SDR systems
- FPGA computing